FORWARD DEPLOYED AI ENGINEERING · CAPSTONE PROJECT

A support assistant for CloudServe Solutions

Classify the ticket, find the right article, draft a cited reply, and hand every reply to a person.

Sushranth Hebbar

Project report · Stage Six submission

PROGRAMME	Forward Deployed AI Engineering
ASSESSMENT	Capstone project (individual)
CLIENT SCENARIO	CloudServe Solutions, customer support function
DOCUMENT	Project report, sections 1 to 10 with appendices A to E
DATE	November 2026
ACCOMPANYING MATERIAL	Video (01_Video), stage workbooks and effort log (03_Workbooks), SupportTicketsResolver repository (04_Source_Code)

Every performance figure in this report states the data it was measured on. The declaration of AI tool use follows Section 10.

Contents

1 Executive summary 4
2 The problem 5
3 Discovery findings 7
4 Requirements 9
5 Architecture and design 11
6 Implementation 13
7 Evaluation 15
8 Governance and risk 20
9 The requirements revision 24
10 Conclusions 26
Declaration of AI tool use 27
Appendix A. Prompt register 28
Appendix B. Full result tables 29
Appendix C. Requirement traceability 31
Appendix D. Workbooks and reproduction 32
Appendix E. Open items and uncertainties 33

Every performance figure states the data it was measured on. Three sets of tickets are used: 500 development tickets (analysis and design decisions), 80 validation tickets (final comparison, never used for tuning), and a hidden set of 120 tickets that I was not given. Appendix E lists what is unmeasured or not yet checked.

Figures and tables

FIGURES

Figure 1 Share of tickets against share of resolution minutes for selected categories 7
Figure 2 The path of a ticket 11
Figure 3 Intent accuracy for the two prompts 17
Figure 4 End-to-end time per ticket on the validation run 18

TABLES

Table 1 The five stakeholders 5
Table 2 Functional requirements 9
Table 3 Non-functional requirements 9
Table 4 The main design choices 12
Table 5 Results on the 80 validation tickets (final configuration, one run) 15
Table 6 The twelve acceptance criteria 18
Table 7 Risk register (likelihood and impact are my own ratings) 20
Table 8 Segment audit on validation (Wilson 95% intervals; variation in points from the best segment) 21
Table 9 Incident procedure 22
Table 10 What changed and why 24
Table 11 The six assumptions 24
Table A1 Prompt register: purpose, version history and the version in use 28
Table B1 Intent classification per class, validation set (80 tickets, final run) 29
Table B2 Routing on validation: route the labels expect against route taken 29
Table B3 Outcomes of the 80 validation tickets 30
Table B4 Latency by outcome, validation run 30
Table C1 Each requirement traced to the code that implements it and the tests that cover it 31
Table E1 Not yet checked 33
Table E2 Estimates, inferences and judgements 33

SECTION 1

Executive summary

CloudServe Solutions has about 150 staff, and its six support agents handle more than 500 tickets a week. A customer waits 8 to 12 hours for a first reply, although the service agreement says two, and about 42% of tickets are resolved on first contact. Marcus Adeyemi, the head of support, asked for a chatbot to answer the easy tickets without adding headcount.

The ticket data pointed somewhere else. In the 500 development tickets, 71.4% are marked answerable from the existing knowledge base. The answers are there. Agents struggle to find them, and escalated tickets reach specialists with no notes on what has already been tried. A chatbot that replies on its own would also run into the one thing Marcus said would sink the project, a confident wrong answer.

So I built an assistant for the agents. It reads a ticket, labels the intent and urgency, pulls the relevant passages from the 29 support articles, and drafts a reply that cites them. Tickets that need a specialist get an escalation summary instead of a draft. An agent reads everything before it goes anywhere, and the code has no way to send a message to a customer. Every step is written to a decision log, and guardrails stop tickets and drafts that carry credentials or instruction-like text.

91.3%
intent correct (73 of 80 validation tickets)

98.1%
answerable tickets with the expected article retrieved (52 of 53)

14 / 14
tickets that must go to a specialist were escalated

80 / 80
tickets processed and recorded in the decision log

55.8 s
95th percentile latency, against a 3 s target

Those results come from the 80 validation tickets, run once in the final configuration. Two other findings shaped the design. Urgency was right for 43.8% of tickets, which is exactly what answering "medium" every time would score, so urgency only orders the agent's queue and never changes a decision. And the model's confidence scores could not be turned into a safe rule for answering automatically, so ordinary tickets go to an agent for review.

> **WHAT THIS REPORT DOES NOT SHOW**
> I did not measure the hallucination rate or citation accuracy, because both need a person to judge each claim. I have no first contact resolution, reply time or satisfaction figures, because the prototype answers no real tickets. The hidden 120-ticket set has not been run by me. The latency figure was taken under free-tier rate limits. The numbers here describe how well the assistant classifies, retrieves and fails safely; they are not evidence that it improves CloudServe's support results.

SECTION 2

The problem

2.1 What CloudServe asked for

CloudServe sells cloud infrastructure and developer tooling to just over 200 corporate customers. Tickets reach its six agents by email, chat, comments on documentation pages and a user forum, and the team is not coping. Marcus Adeyemi reports a first reply time of 8 to 12 hours against a two-hour agreement, first contact resolution of 42% against an industry figure he puts at 65%, and customer satisfaction that has dropped to 3.2 out of 5. He asked for a chatbot that handles the easy tickets so that response times fall without new hires. He was equally clear about what would make the project fail: wrong answers, more work for his agents, worse service for enterprise customers, and decisions nobody can explain.

The brief gives four more voices, in interview transcripts. Each of them sees one part of the operation, and Table 1 sets out what they said and what they could not see.

Table 1. The five stakeholders

Person	What they told me	What they could not see
Marcus Adeyemi Head of support	First contact resolution matters more to him than reply time. Wants the easy tickets answered. Fears wrong answers and decisions he cannot explain.	The ticket mix. He says any breakdown would be a guess.
Sofia Restrepo Tier One agent	Handles 40 to 70 tickets a day. Familiar ones take 4 to 5 minutes, unusual ones up to 40. Documentation search is so poor that agents keep personal answer files. Would welcome a draft with its article attached. Thinks non-fluent customers fare worst.	Her figure of 70% familiar tickets is experience, not a count.
Daniel Okonkwo Tier Two engineer	About half of what reaches him could have been settled at Tier One. Escalations arrive with no summary, so he rereads the thread and repeats questions.	"About half" is an impression, not a measurement.
Ines Varga Technical writer	Keeps 29 articles she considers accurate. Agents rarely use them because search matches titles and exact words. Wants any system to name its source.	How often the articles are used, and how many tickets they cover.
Ravi Menon Customer	Support is slow but good once reached. Waiting hurts most when a deployment is blocked. Finds the answer himself about half the time. Would accept a clearly labelled machine draft that a person has confirmed.	Whether his experience is typical.

2.2 What the data showed

The documentation is not where the gap is. In the 500 development tickets, 71.4% are labelled answerable from the existing articles. More than half of all tickets (56.2%) were escalated, and 49.1% of the escalated ones were answerable from documentation, which lines up with Daniel's "about half". CloudServe holds most of the answers but cannot put them in front of the right person quickly, and it loses time a second time when a ticket does need a specialist, because the hand-over carries no context.

THE GAP

CloudServe asked for automation. The evidence supports help with finding and drafting: locate the article, draft a cited reply, say when there is nothing to go on, and pass on what has been tried. It also says the cost of a mistake is high, since the head of support himself lists a wrong answer as a way the project fails. Both points argue for an assistant whose work an agent checks, not one that writes to customers by itself.

Three things stayed out of scope from the start. The project does not rewrite the knowledge base, because retrieval already finds the right article for nearly every answerable ticket. It does not replace any agent. And it must not widen the service gap Ravi worries about, so every result in this report is also looked at by customer group (Section 8).

SECTION 3

Discovery findings

The evidence came from the five transcripts and from the 500 development tickets, each of which carries labels and a record of how people handled it. Every figure below comes from those tickets. The full tables are in the Stage 1 workbook.

3.1 The answers exist

357 of the 500 tickets (71.4%) are labelled answerable from the current articles. Historically, 56.2% were escalated, and 49.1% of those escalated tickets were answerable from documentation. This is the finding that moved the design away from a model that answers from its own knowledge. The system retrieves an article and cites it. In this sample first contact resolution is 43.8% and average satisfaction 2.97 out of 5, against the 42% and 3.2 that Marcus reports. The gap could come from different periods or definitions, and I treat Marcus's figures as the operating baseline.

3.2 The minutes go to a few kinds of ticket

Resolution minutes are my measure of effort. The average ticket takes 421.7 minutes. Security incidents average 785.9 minutes and make up 5.2% of tickets but 9.7% of all minutes. Feature requests average 723.9 and compliance requests 679.3. The cheap tickets are the routine ones: rate limits average 122.2 minutes, single sign-on configuration 191.0 and onboarding 202.1. Figure 1 sets each category's share of tickets against its share of minutes.

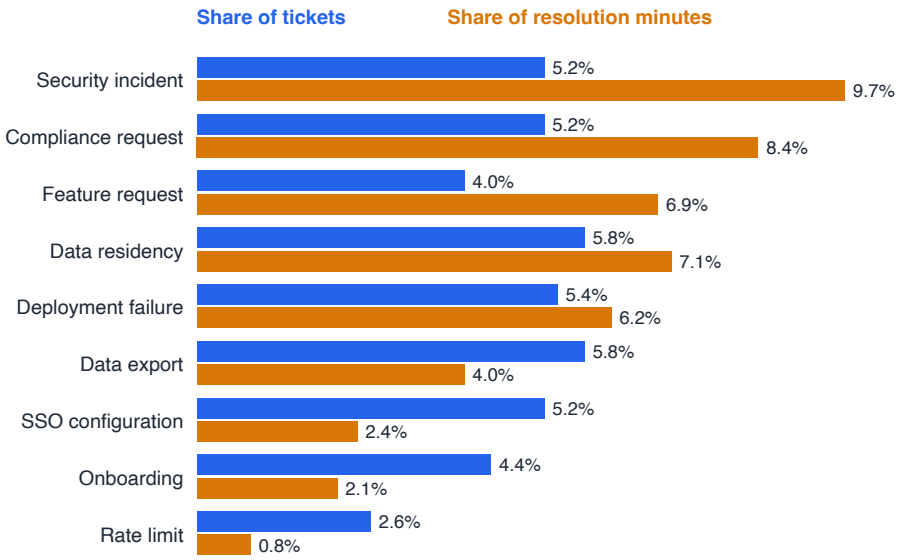


Figure 1. Share of tickets against share of resolution minutes for selected categories (500 development tickets; effort is the sum of historical resolution minutes).

The pattern suggested a division of labour. Routine, well-documented categories are where a draft saves an agent time. Security, compliance, feature requests and unclear tickets are where a specialist is needed and where the corpus may have no answer at all, as Ines pointed out for feature requests and new incidents. I turned that into a standing rule: those four intents always escalate, whatever the model's confidence.

3.3 First contact resolution and satisfaction move together

Marcus put first contact resolution first and Ravi put waiting first. Satisfaction averages 3.41 for tickets resolved on first contact and 2.63 for the rest. It averages 3.40 for the fastest quarter of tickets and 2.63 for the slowest. The two measures overlap, since a ticket resolved first time is also a quick one, so I kept both as goals and watch satisfaction so that it does not slip while resolution improves.

3.4 The non-fluent customers

Sofia said non-fluent customers take longer and are the least satisfied. Half of that holds up. The 120 non-fluent tickets (24.0%) have a mean resolution time of 466.1 minutes against 407.7 for fluent customers. Their satisfaction, though, is 3.04 against the 2.97 average, and their first contact resolution of 45.8% is slightly above the 43.8% overall. The cost shows up as time, not as unhappiness. I treated it as a fairness risk for retrieval, which depends on a customer's wording matching the documentation, and tested it directly (Section 8).

3.5 Enterprise customers do worst

Ravi worried about a gap between business and enterprise customers. The data shows the gap, in a direction I did not expect. Enterprise tickets have the lowest first contact resolution (37.3%, against 48.2% for business and 43.1% for standard), the highest escalation rate (62.7%) and the longest mean resolution time (483.7 minutes). Any change that does not make this worse has to be checked on enterprise tickets in particular.

3.6 What nobody mentioned

Nobody raised the handling of customer information, although every ticket carries a name and an ID. That became the privacy and auditability requirements. Nobody raised differences between channels either. When I checked later, the differences I found mostly disappeared once I accounted for how near-duplicate tickets cluster (Section 7).

> **PROBLEM STATEMENT, AS WRITTEN AT THE END OF DISCOVERY**
>
> CloudServe's support team is struggling to keep up with more than 500 tickets a week. Customers wait 8 to 12 hours for a first reply, despite the two-hour service promise, and fewer than half of tickets are resolved on first contact. Many answers already exist in the support articles, but agents struggle to find them, and escalations often arrive without clear notes on what has already been tried. Customers wait longer, and agents repeat work.

SECTION 4

Requirements

I wrote the requirements document at the end of the first week and revised it after the build. The current version is 2.0, dated 11 November 2026, and version 1.0 of 31 October stays in the repository history. Each requirement has an identifier, a priority and the discovery finding it came from, and Appendix C (Table C1) ties each one to code and tests. Table 2 lists the functional requirements and where each stands, and Table 3 the non-functional requirements.

Table 2. Functional requirements

ID	Requirement	Priority	Where it came from	Status
FR-01	Accept a ticket and show its available context to the agent	Must	Sofia's queue; Daniel rereading threads	Built
FR-02	Find and show the relevant documentation	Must	71.4% answerable; Sofia and Ines on search	Built. Plan-scope check specified, not built
FR-03	Draft a reply grounded in the shown documentation	Must	Sofia and Ravi want a source and a draft	Built. Subject-versus-body rule specified, not built
FR-04	Agent reviews, edits, rejects or approves before anything is sent	Must	Marcus on wrong answers; Ravi on human confirmation	Built. No send path exists
FR-05	Recognise insufficient evidence and recommend clarification or escalation	Must	Marcus, Sofia, Daniel	Built, with the four must-escalate intents
FR-06	Prepare an escalation summary	Should	Daniel on missing context	Built
FR-07	Record sources used and the agent's decision	Must	Marcus and Ines on traceability	Built
FR-08	Capture outcomes and agent feedback	Should	First contact resolution as the key measure	Built. No pilot data yet
FR-09	Label intent and urgency, each with a confidence	Must	Marcus has no ticket breakdown; spec A3	Built. Urgency advisory only

Table 3. Non-functional requirements

ID	Area	Requirement
NFR-01	Latency	95th percentile under 3 seconds end to end.
NFR-02	Availability	Available in support hours with a recovery path. Targets still to be agreed.
NFR-03	Accuracy	Hallucination rate 5% or lower and citation accuracy 95% or higher, established by human review. Automatic citation checks are retrieval checks only.
NFR-04	Privacy	One customer's information never appears in another's ticket or reply.
NFR-05	Auditability	Any assisted reply can be traced to its sources and the agent's decision, with access controlled.
NFR-06	Fairness	A gap of under 5 percentage points between a customer group and the rest, estimated by resampling whole scenarios. Groups under about 25 tickets are inconclusive.

ID	Area	Requirement
NFR-07	Cost	Within an agreed limit per assisted ticket. Limit still to be agreed.
NFR-08	Evaluation integrity	Supplied labels are never rewritten. Calibration is split by scenario. Prompt and policy choices are made on development data only. (Added in version 2.0.)

4.1 What is deliberately out of scope

The PRD lists what it does not build and why. There is no autonomous chatbot, because the head of support's worst case is a confident wrong answer and the customer wants a person to confirm. Agents are not replaced. The knowledge base is not rewritten, since most answerable tickets already have an article that retrieval finds. Agents' personal answer files are not used as sources, because Daniel warns they may be wrong. Tiers, service agreements and staffing are CloudServe's decisions.

4.2 Assumptions written down in advance

Version 1.0 recorded three assumptions: the documentation is usable as an answer source, tickets contain enough context for a useful draft, and agents will review drafts properly. Version 1.0 also rested on three that I had not written down, namely that a confidence score can decide which tickets get an automatic answer, that urgency can be predicted from the text and used to route, and that a free model provider can run the evaluation unattended. Section 9 reports how each of the six turned out.

4.3 Success measures

The measures in the PRD fall into two groups. The offline evaluation can report intent precision and recall, retrieval hit rate, a citation proxy, latency and group gaps. First contact resolution, time to first reply and satisfaction need a supervised pilot with real outcomes, so the evaluation reports them as unavailable and never fills them from the historical baseline. That is why this report cannot say whether the assistant improves CloudServe's first contact resolution.

SECTION 5

Architecture and design

5.1 How a ticket moves through the system

A ticket from any of the four channels is first turned into one common format. Customer fields are held apart, and the evaluation labels and historical outcomes are kept out of everything the model sees. A language model then classifies the ticket: one of 22 intents, an urgency, a confidence for each, and for urgency a short quotation from the ticket as evidence. A fixed router decides what happens next. If the intent is a security incident, compliance request, feature request or unclear request, the ticket goes to a specialist, and the system retrieves the closest articles and prepares an escalation summary. Every other ticket gets its five closest passages retrieved, then a draft written by the model from those passages only, then guardrail checks on the ticket and the draft. What comes out is either a draft waiting for an agent, or a note to a human saying why no draft was possible. Each step adds a row to the decision log. Figure 2 shows this path.

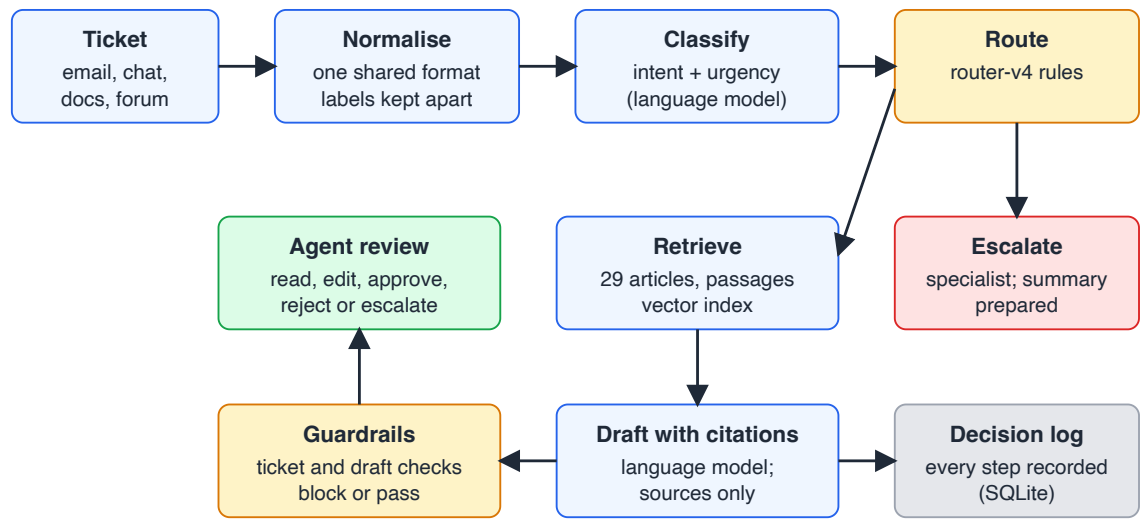


Figure 2. The path of a ticket. Everything ends at the agent's review screen and nothing is sent to a customer.

Retrieval covers the 29 supplied articles. Each is cut into passages of about 800 characters with a 120-character overlap, embedded with the MiniLM sentence model and kept in a persistent vector index. The drafting prompt tells the model that ticket text is untrusted data, allows it to use only the retrieved passages, and requires an inline citation such as [DOC-AUTH-004] after each claim. Agents work in a small web page where they can read a draft and its sources, edit it, approve it, reject it or escalate it, and approval runs the citation and safety checks again. A separate store holds observed outcomes and agent feedback, and a dashboard summarises decisions, guardrail activations and

outcomes by customer tier and fluency, with a plain statement that it makes no causal claim. The language model was Qwen 3.8 27B served by Groq on its free tier. The classification prompt is PR-03 and the drafting prompt is PR-02; Table A1 in Appendix A lists every prompt and its versions.

5.2 Choices and alternatives

Table 4 sets out the main design choices, the alternative considered for each, and the reason for the choice.

Table 4. The main design choices

Choice	Alternative	Why
Draft for agent review; no send path	A chatbot that replies directly	The head of support's worst case is a wrong answer, and the customer wants a person to confirm. The automatic checks cannot tell whether a claim fits the ticket, so a person has to.
Answer only from retrieved passages, cite inline	Let the model answer from what it knows	71.4% of tickets are answerable from the articles, and Ines wants any error traceable to the article or the system.
Four intents always escalate	Gate every ticket on calibrated confidence	Confidence did not separate right answers from wrong ones (Section 7.4). The rule is simple, and the supplied labels follow the same rule.
Urgency is advisory	Escalate high-urgency or ambiguous tickets	Urgency accuracy equalled always answering "medium", and a high urgency does not mean a hard ticket.
Escalation summary built from the ticket, its history and the sources	Have the model write it (prompt PR-04)	A model could invent troubleshooting that never happened. The built summary flags missing context as missing.
Calibration split by scenario	Split by ticket text	The text split leaked: 87 of 90 held-out tickets had a paraphrase in the fitting set.
Groq free tier	OpenRouter free tier	The first provider throttled the runs; with Groq plus pacing and retries the full set could finish.
SQLite decision log	A hosted database	No cost and no extra service to start from a clean checkout.

5.3 What it does not have

There is no live intake. Tickets enter through the evaluation runner, the scripts and the tests, and nothing reads a real support inbox. The system does not measure agent time and does not detect duplicate tickets, although the data has many near-duplicates (Section 9). It depends on a single model provider, and I tested what happens when that provider fails (Section 8.5) without removing the dependency.

SECTION 6

Implementation

6.1 How it was built

The repository was created on 26 October 2026. The first week went on the discovery workbook, the requirements, the prompt library and the sprint plan. In the second week I built the normaliser, the evaluation harness, the classifier and router, retrieval, grounded drafting with its guardrails, the escalation summary, the agent review page, outcome capture and the dashboard. The third week is for governance, this report and the video. The code has one module per job (*tickets*, *classification*, *routing*, *retrieve*, *answering*, *guardrails*, *escalation*, *pipeline*, *evaluation* and the stores), so each can be tested alone. The tests run with *python -m pytest tests -v*. Under Python 3.11 the full suite of 68 tests passes.

The evaluation harness takes an input path and an output path as arguments, because the final assessment runs it on a file I have not seen. A failing ticket is recorded as an error and the run continues. The harness writes one result row per ticket, builds a metrics report, and can resume an interrupted run by reusing the tickets already done. The resume feature turned out to be essential.

6.2 What was hard

The model provider. My first full run, on 1 November, finished 6 of 80 tickets before the free provider began answering HTTP 429, "too many requests". After moving to Groq on 2 November the first attempt reached 18 tickets. I added pacing between requests, retries that wait for the reset time the provider states, and the resumable run. The final validation run still took two sessions on two days because of the daily token allowance, so the criterion of a single unattended run is not strictly met on the free tier.

Citations. My first drafting prompt let the structured citation list and the inline identifiers disagree, which would have thrown away sound drafts. Version 2 of the prompt (3 November) requires them to match, and the stored citations are taken only from the inline identifiers, each checked against the passages retrieved for that ticket. A small probe on development tickets showed the failure that remains: a draft can cite a real document that does not back its sentence, and the check cannot see that. For that reason I call it a retrieval check.

Evaluation method. The hardest problem was not a bug. My first calibration split was by text hash, and nearly every held-out ticket had a paraphrase in the fitting set. I replaced it with a split by scenario, grouping near-duplicates by shared content words (about 79 groups in the development set). The same clustering showed that intervals computed on single tickets were far too narrow, so every interval in this report resamples whole scenarios. Separately, 67 of 97 groups of identical ticket text carry different urgency labels. I report that conflict and did not relabel, because the hidden set may conflict in the same way.

Smaller problems. The vector store printed telemetry errors and reloaded the embedding model for every ticket, which I fixed by pinning a dependency and loading once. The pipeline reported its latency as empty, so the first report had no latency figures; I changed the report to use the harness's own

timing and added a test. A malformed ticket in an input file stopped the whole run, which I found while testing failure handling (Section 8.5) and fixed.

6.3 What I would change

I would add a live intake, so that the fallback which sends a ticket to review when classification fails is something a real queue uses; at the moment only the tests and a demonstration call it. I would have the pipeline record where each ticket's time goes, waiting and retrieval and each model call, which it does not do now. I would keep the prompts together with their versions in one place instead of partly inside the code. And I would build the two draft fixes described in Section 9, which need a fresh, unseen set of tickets to evaluate because the validation set has been used.

SECTION 7

Evaluation

7.1 Method and data

Three sets of tickets serve three purposes. The 500 development tickets were for analysis, fitting calibration and choosing between prompts and rules. The 80 validation tickets were held back for the final comparison and used for no tuning. The hidden 120-ticket set has not been given to me, so I have run the system on it zero times; the harness takes any input path and writes the same report, so the course can run it.

The validation run in this section used the final configuration: classification prompt PR-03 version 4, router version 4, drafting prompt PR-02 version 2, and confidence calibration switched off. All 80 tickets were processed with no errors. The free provider's daily allowance split the run into two resumed sessions, and it was complete by 10 November 2026. The report file was regenerated on 11 November to fill in a latency block that had come out empty.

Paraphrases of one scenario are close to identical, so tickets are not independent. Every interval here is a 95% interval from resampling whole scenarios, 2,000 times (50 scenarios in the validation set, 76 in the development predictions). Resampling single tickets would give intervals about half as wide. Where two systems are compared on the same tickets I use the paired difference.

7.2 Business outcomes

I cannot report the outcomes CloudServe cares about. First contact resolution, reply time and satisfaction need real tickets answered with and without the assistant, and the prototype answers none, so those fields in the report are empty by design. For context, the validation tickets' own history shows what human handling achieved: first contact resolution of 46.3%, escalation of 53.8%, repeat contact of 21.3%, a mean resolution time of 441 minutes and satisfaction of 2.6 out of 5. That is the baseline a pilot would have to beat. It is not a result of the system.

7.3 Technical results

Table 5 gives each technical measure with its target and the way it was measured.

Table 5. Results on the 80 validation tickets (final configuration, one run)

Measure	Result	Target	How it was measured
Intent accuracy	91.3% (73 of 80); interval 83.3 to 97.5	85% precision per class	Exact match with the supplied label, 22 classes
Intent precision and recall, macro	0.94 and 0.92	85% precision per class	Per class. Four classes fall below 85% precision, on 3 or 4 tickets each (Table B1)
Must-escalate tickets caught	14 of 14	all	Predicted intent is one of the four must-escalate intents
Urgency, exact	43.8% (33.3 to 54.9)	advisory	Always answering "medium" scores 43.8% (35 of 80)

Measure	Result	Target	How it was measured
Urgency, within one level	85.0%	advisory	With three levels this is uninformative alone
Retrieval hit, answerable tickets	98.1% (52 of 53)	none set	An expected article is among the five retrieved
Citation match (proxy)	78.7%	95% accuracy	The cited document is an expected one. A proxy, not citation accuracy
Latency, median and 95th percentile	6.3 s and 55.8 s	p95 under 3 s	End to end per ticket, with request pacing and rate-limit retries inside the time
Hallucination rate; citation accuracy	Not measured	5% or lower; 95% or higher	Needs human review, which I did not carry out
Decision log reconciliation	80 of 80	exact	80 classification and 80 routing rows; 63 drafting and 17 escalation rows

The classifier does best where the classes are distinct and worst where two labels overlap. On validation the weak spots are API key issues, with 2 of 6 found and most of the misses read as security incidents (which escalate, so the failure is a safe one), and three small classes: account access, authentication failure and quota or overage. With 3 to 6 tickets in each, the per-class figures in Table B1 are indications, not estimates.

7.4 Changing the prompt

The first classification prompt (PR-03 version 2) reached 85.6% intent accuracy on the development predictions (interval 77.2 to 92.8). Errors were concentrated: 37 of the 49 fell in 8 of 76 scenarios. One was a compliance request, an auditor asking for access records, read as a data export, which would have been answered when it should have been escalated. I wrote version 4 of the prompt with explicit intent definitions and tie-break rules. Before running it I wrote down a hypothesis, H1: intent accuracy improves, no class loses more than 10 points, and macro recall rises.

On the development tickets accuracy rose to 93.2%, a paired gain of 7.6 points (interval 1.6 to 14.7), with 30 tickets fixed and 4 broken. The must-escalate tickets caught went from 56 of 64 to 64 of 64. Figure 3 compares the two prompts on both sets. H1, however, was not met, because recall for authentication failure fell by 21 points. I recorded that verdict as written and adopted the prompt anyway. The class that lost is a pair of labels that overlap and both routes end with a person, and the gains were larger. It was a decision, not a pass, and the revision log says so.

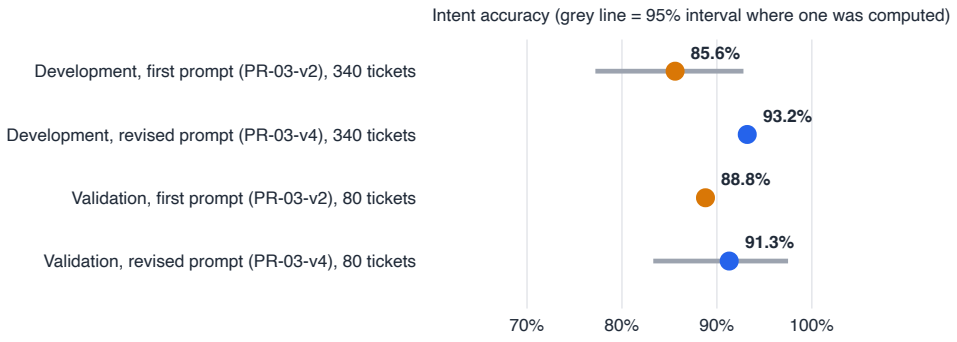


Figure 3. Intent accuracy for the two prompts. On validation the prompts differ by two tickets (71 against 73 of 80), which is within noise; the development gain is the evidence for the change.

I used the validation set once, for the final prompt, and tuned nothing on it. That also means I have no unseen data left on which to test another prompt change.

7.5 Confidence, calibration and routing

I had hoped to use the model's own confidence as a gate, answering only above a calibrated 0.80. With this model that does not work. PR-03 version 4 produces only three confidence values for intent: 0.85, 0.95 and 0.98. On the development predictions, intent was right for 94.0% of the 151 tickets at 0.98 and 92.6% of the 188 at 0.95, a gap of 1.4 points. On validation the gap is wider (97.6% of 41 against 84.2% of 38) but rests on few tickets, and since every value sits above 0.80 a gate at that threshold would remove nothing either way. I fitted three isotonic calibrations, split by scenario. The latest is flat, and none could be shown to separate right answers from wrong ones. Calibration stays off, which sends every ordinary ticket to review.

That explains the routing numbers. The supplied labels expect 48 of the 80 validation tickets to be answered automatically and 32 to be escalated. The system escalated 17 and sent 63 to an agent, so strict routing accuracy is 17.5% (14 of 80; Table B2 in Appendix B). Of the 32 labelled for escalation, 14 were escalated and 18 went to review, where an agent sees them. Nothing was answered automatically. I report the figure as it is: the prototype chooses not to automate.

7.6 Retrieval and the drafts

Retrieval worked well. On development an expected article was among the five retrieved for 95.8% of answerable tickets, and was the top result for 88.8%. On validation it was among the five for 98.1%. Of the 63 tickets that reached drafting, 50 produced a cited draft and 13 returned a note that the evidence was insufficient (Table B3).

Reading the drafts against the cited articles showed what the automatic checks miss. All 50 drafts passed citation validation, yet some were wrong for the ticket. VAL-0049 had the subject "Need to rotate an exposed key" over a body describing a read-only scope error; the draft told the customer to revoke the key as exposed. VAL-0078 had a subject about a slow database restore over a body about a connection-pool error, and the draft discussed restore times the customer never mentioned. VAL-0051

and VAL-0076 were Standard-plan customers told about features that the cited articles limit to Business and Enterprise plans. These examples are the basis for two changes to the requirements in Section 9.

7.7 Latency

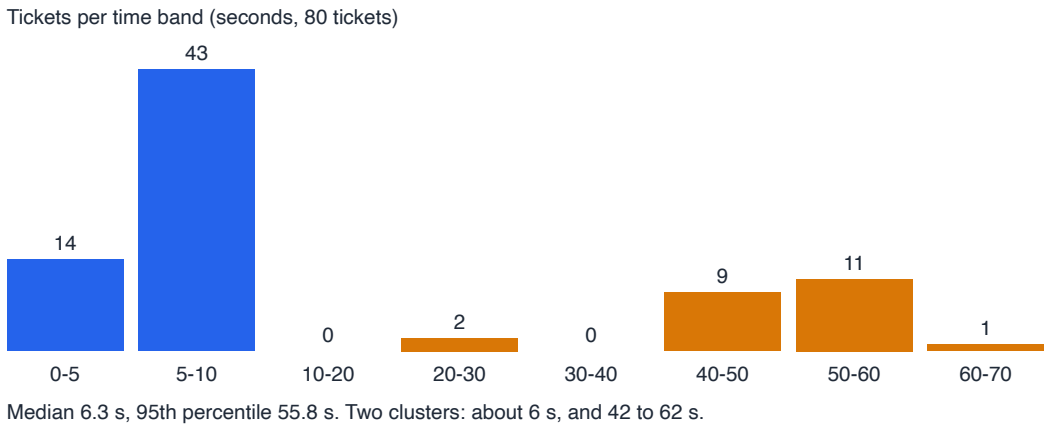


Figure 4. End-to-end time per ticket on the validation run. The 21 tickets between 42 and 62 seconds appear in every kind of outcome, including escalations that make a single model call.

Figure 4 shows the time taken for each ticket, and Table B4 breaks it down by outcome. The median was 6.3 seconds and the 95th percentile 55.8 seconds, which misses the 3-second target. Fifty-seven tickets finished in under 10 seconds and 21 took between 42 and 62 seconds. The run used a rate-limited free tier with request pacing and retries inside the measured time, so these are end-to-end times under those conditions and not a production figure. A fair measurement needs an unpaced run on a key with higher limits, which I did not make.

7.8 Acceptance criteria

Table 6 gives the status of each of the twelve acceptance criteria in the Build Specification.

Table 6. The twelve acceptance criteria

#	Criterion	Status
A1	Runs from a clean checkout	Not yet rehearsed (Appendix E)
A2	Four channels normalised	Met. Validation includes all four channels, handled by one normaliser, with tests
A3	Intent, urgency and confidence on every ticket	Met. 80 of 80, each with numeric confidences
A4	Retrieval returns identifiable passages	Met. Sources are document identifiers that resolve to the supplied articles
A5	Deterministic routing	Router is a pure function of the classification; live repeat test not run (Appendix E)
A6	Citations resolve to retrieved passages	Met at retrieval level for all 50 drafts

#	Criterion	Status
A7	A guardrail blocks	Met. Four engineered tickets were blocked live with a logged block (Section 8.4)
A8	Decisions logged and reconciled	Met. 80 tickets, 80 classification and 80 routing rows
A9	Single unattended run	Not strictly met. Two resumed sessions on the free tier
A10	Metrics report produced	Met. Business and hallucination fields are empty by design
A11	Failure handled	Met for provider disconnection, rejected key, timeout, rate limit, empty retrieval and malformed input (Section 8.5)
A12	Tests pass with one command	Met. All 68 tests pass under Python 3.11 with the documented command

7.9 What I did not measure

I did not measure the hallucination rate or citation accuracy, because that needs two people to judge the drafts and I did not arrange it. I did not measure first contact resolution, reply time or satisfaction, since there was no pilot. I did not run an unpaced latency test, a load test or a cost per ticket. I did not run the hidden set. Groups of 8 to 61 tickets cannot show a gap smaller than about 20 points, so "no evidence of a gap" in Section 8 is a weaker statement than "no gap".

SECTION 8

Governance and risk

The governance framework offers one test: pick a way the system could harm a customer and ask what in the design stops it. Where my answer was only a property of the model, I have said so and not counted it as a control. The full governance document is *docs/governance.md* in the repository.

8.1 Decision logging

Every ticket writes a classification row and a routing row, plus either a drafting row or an escalation-summary row, and agent actions add a review row. The log holds every field in the framework's minimum record. On the final validation run it reconciles exactly: 80 tickets, 80 classification rows, 80 routing rows, 63 drafting rows and 17 escalation rows, so every ticket has a draft or a summary and none has both. Three weaknesses remain. The list of alternatives is never filled, because the classifier returns one label. The drafting row stores the full ticket text and passages so that the review page can show them, which means access to the database needs a retention rule that CloudServe has not set. And before my coverage check a classifier failure left no row at all; it now writes an error row.

8.2 Risk register

Table 7 lists the eight risks I assessed, with the mitigation for each and what that mitigation does not cover.

Table 7. Risk register (likelihood and impact are my own ratings)

ID	Risk	Likelihood / impact	Mitigation, and what it does not cover	Owner
R-01	Confident, wrong answer	Medium / High	No send path; every draft reviewed; answers cite retrieved sources; four intents always escalate. Not covered: the automatic checks pass drafts like VAL-0049, and the hallucination rate is unmeasured.	Sushranth Hebbar
R-02	Private data in an outbound reply	Low (nothing is sent) / High	Tickets and drafts are blocked if they contain key-, token- or password-like strings. Not covered: names, emails, addresses, account numbers.	Sushranth Hebbar
R-03	Customer text treated as an instruction	Medium / Medium	The prompt marks ticket text as untrusted; ticket and draft are scanned for instruction-like text. Not covered: paraphrased attacks; no injection test set was run.	Sushranth Hebbar
R-04	Some groups get worse answers	Medium / Medium	Group audit in 8.3. Not covered: draft correctness and tone by group.	Sushranth Hebbar
R-05	Documentation goes stale	High over time / Medium	Each citation names its article. Not covered: no freshness check; the plan-scope check is specified, not built.	Technical writer (proposed)
R-06	Model provider unavailable	High on free tier / Low here	Retries, safe results, logged errors, a resumable run, and an option that sends the ticket to review. Not covered: no second provider.	Sushranth Hebbar
R-07	Latency under load	High / Medium	Measured in 7.7 and over target. No load test.	Sushranth Hebbar

ID	Risk	Likelihood / impact	Mitigation, and what it does not cover	Owner
R-08	Cost grows with volume	Unknown / Medium	A drafted ticket makes two model calls, an escalation one. Not covered: no cost measured or capped.	Support Operations (proposed)

8.3 Fairness audit

I compared the segments the framework lists on the 80 validation tickets. The quality score is intent accuracy. Resolution rate is unavailable because the prototype resolves nothing. The intervals in Table 8 treat tickets as independent, so they are narrower than the scenario-based intervals elsewhere.

Table 8. Segment audit on validation (Wilson 95% intervals; variation in points from the best segment)

Segment	Tickets	Intent accuracy	Variation	Retrieval hit	Draft ready
Enterprise	8	88% (53 to 98)	–7	100% (6/6)	88%
Business	30	90% (74 to 97)	–5	100% (22/22)	73%
Standard	42	93% (81 to 98)	–2	96% (24/25)	50%
Fluent English	61	90% (80 to 95)	–5	98% (43/44)	62%
Non-fluent English	19	95% (75 to 99)	best	100% (9/9)	63%
Short (up to 24 words)	43	93% (81 to 98)	–2	96% (27/28)	65%
Long (over 24 words)	37	89% (75 to 96)	–6	100% (25/25)	59%

No intent or retrieval gap stands out from the noise. The framework expects retrieval to do worse for non-fluent English, and here it did not: those tickets scored best on intent and retrieved the expected article in all 9 answerable cases. With 19 tickets I could not see a gap smaller than about 20 points. The draft-ready rate varies by tier (88%, 73%, 50%), and the reason is the mix of tickets: 11 of the 14 must-escalate tickets in the set are standard tier, and those never get a draft. Enterprise has only 8 tickets, so its row proves nothing either way. On the development set, chat and Europe looked 10 points worse on intent, but both gaps traced to a few scenarios and did not appear on validation, where chat matches the rest and Europe is 7 points higher. I did not test whether drafts differ in correctness or tone by group.

8.4 Guardrails and the demonstration

The framework names five guardrails. The confidence floor is in place: a missing or out-of-range score goes to review. Private data and instruction integrity are partly covered, since they scan the ticket and the draft for credential-like strings and phrases such as "ignore the previous instructions" and nothing more. Grounding is partly covered: it requires an inline citation to a retrieved document and does not test the claim. Tone and scope, which should stop the system promising a refund or a deadline, is not built. When a guardrail fires, the draft is blocked, the ticket goes to review and the block is logged.

The validation run produced no block, so I tried to provoke one. Seven tickets built to make the model repeat a credential or write an injected sentence all failed, because the model refused or pointed out the credential. That is good behaviour, but it belongs to the model and not to a control I wrote, so it

does not satisfy criterion A7. I added a check on the ticket itself. If the subject or body contains a credential-like or instruction-like string, the pipeline skips drafting, writes a block decision naming the category and not the secret, and routes the ticket to review. Four engineered tickets sent live through the real pipeline and model were all blocked, with a logged block and no draft. The same patterns match none of the 580 development and validation tickets, so the check would not have changed an earlier result, although the validation run predates it.

8.5 Failure handling

I induced each failure the specification names. With the provider's address pointed at a closed port, a two-ticket run finished and logged both errors; a disconnection during drafting produced a note for a human to retry later. A rejected key, a timeout and rate limiting (the last two simulated, with retries) gave the same safe result. An empty retrieval and a retrieval crash each produced a note for review. One malformed ticket in a file of three stopped the whole run, which was a defect; the harness now skips invalid records, lists them and processes the rest. When classification is unavailable, an option sends the ticket to review with no intent and writes a routing row, so the log still reconciles. Nothing in the prototype uses that option yet, since there is no live intake.

8.6 Incident procedure and kill switch

Table 9 sets out the incident procedure step by step.

Table 9. Incident procedure

Step	What to do	Who
1. Detect	An agent reports a wrong or unsafe draft, the dashboard shows more blocked or insufficient-evidence outcomes, or the decision log stops reconciling with tickets processed. Note the ticket ID.	Any agent; the owner reviews the dashboard daily in a pilot
2. Contain	Unset the calibration setting if it is set, so every ticket goes to review, and stop the service. Do not delete rows.	Owner
3. Assess	Read the ticket's classification, drafting and routing rows, with prompt version, sources and guardrail results. Decide whether the cause was the model, retrieval, a stale article, a guardrail gap or tampering, then look for similar tickets.	Owner
4. Notify	Tell the head of support and the affected agents the same day. If a customer was told something wrong, the agent corrects it through the normal channel.	Owner to head of support
5. Remediate	Fix the cause, add a test that reproduces the ticket, run the suite, then restore.	Owner
6. Review	Record what happened, update the register, and check whether the requirements need revising.	Owner with head of support

The kill switch is the weakest part of the design. There is no dedicated switch. The system cannot send to a customer, and automatic routing is off whenever the calibration setting is unset, but turning anything off means editing the environment file and restarting the service. That takes about a minute, applies from the next start and not the next ticket, and is open to anyone who can edit the file. I have not tested restart-to-stop as a switch. Before any real deployment I would add a flag read on every ticket, with a named operator.

DECLARATION, IN THE FRAMEWORK'S WORDS

This system must never send a message to a customer without an agent approving it, or answer a security incident, compliance request, feature request or unclear request without a specialist. **What enforces that:** the code has no send path, approval is a separate agent action that runs the citation and safety checks again, and a routing rule escalates those four intents whatever the confidence (14 of 14 on validation, 64 of 64 on development). **The most likely way it could still do harm:** a fluent, cited draft that gets one claim wrong or applies an article to the wrong plan, which a busy agent approves. **We would not deploy it without:** a two-assessor review of at least 50 drafts, a supervised pilot, the plan-scope and commitments checks, a tested runtime kill switch, and agreed rules for data access and retention.

SECTION 9

The requirements revision

I revised the requirements on 11 November 2026 and wrote up each change in the revision log: what changed, what prompted it and who agreed. I was the only author and reviewer, so the log names me, and the document says it is not yet approved by anyone else. Eight requirements changed, one was added, none removed, and the success measures were rewritten (Table 10).

Table 10. What changed and why

Requirement	Change	What prompted it
FR-02 documentation	Tell the agent when an article's "applies to" scope does not match the customer's plan (specified, not built)	VAL-0051 and VAL-0076: Standard-plan customers told about features limited to Business and Enterprise
FR-03 grounded draft	Answer what the ticket body says and treat a conflicting subject as unconfirmed (specified, not built)	VAL-0049 and VAL-0078, where the draft followed the subject line
FR-04 agent review	The system has no send path; "auto-respond" means only that a draft is ready	The route name was read as "sent automatically" and confused the routing numbers
FR-05 insufficient evidence	Four intents always escalate	With the first prompt, 7 of 17 compliance requests were read as data exports and would have been answered
FR-09 classification	Intent confidence gates drafting only once calibrated; urgency is advisory with no accuracy target	Urgency matched always-"medium"; confidence did not separate right from wrong
NFR-01 latency	Target fixed at p95 under 3 seconds; first measurement recorded as missed	The "to be decided" had to be settled to evaluate against anything
NFR-03 accuracy	Hallucination 5% or lower and citation accuracy 95% or higher by human review; automatic checks called retrieval checks	All 50 drafts passed the automatic checks, including the wrong ones above
NFR-06 fairness	Gap under 5 points, intervals from scenario resampling, small groups inconclusive	Per-ticket intervals were too narrow because paraphrases are near-duplicates
NFR-08 (new)	Labels never rewritten; calibration split by scenario; decisions on development data only	My first split leaked: 87 of 90 held-out tickets had a paraphrase in the fitting set
Section 8 measures	Split into offline measures and pilot-only measures	The prototype never sends, so resolution, reply time and satisfaction cannot be measured offline

9.1 How the assumptions turned out

Table 11 sets each of the six assumptions from Section 4.2 against the evidence.

Table 11. The six assumptions

Assumption	Held?	Evidence
The documentation is usable as the answer source	Largely	Expected article retrieved for 52 of 53 answerable validation tickets. The weakness was applicability to a plan, not coverage

Assumption	Held?	Evidence
Tickets carry enough context for a useful draft	Partly	Bodies are usually enough, but subjects often disagree with them and the drafter followed the subject in the cases above
Agents will review and edit drafts properly	Not tested	There was no pilot
A confidence score can decide which tickets get an automatic answer (unwritten)	No	Mean intent confidence was 0.958 when right and 0.934 when wrong on development
Urgency can be predicted from the text and used to route (unwritten)	No	Exact accuracy 47.5% on development, 43.8% on validation; level with always-"medium"
A free provider can run the evaluation unattended (unwritten)	No	HTTP 429 stopped early runs; the final run needed two sessions

9.2 What it taught me

I started from the idea that confidence and urgency would work as control signals: a score would say which tickets could be answered and urgency which needed a person. The data said otherwise. What the interviews actually asked for was help finding the right article and a draft an agent can check, so the real job was making the agent's review dependable, not deciding what to automate. I would also look at the data before writing a target. Counting the repeated-text groups in the first week (67 of 97 with conflicting urgency labels) and comparing against the always-"medium" baseline would have shown that an urgency target meant nothing, and a small calibration check on 50 tickets would have done the same for confidence. I did neither until the second week.

Several things I chose not to change are in the log with the reason. I did not relabel the conflicting urgency groups, because the hidden labels may conflict in the same way. I did not switch calibration on, because the gate filtered nothing. I did not rewrite the knowledge base, because retrieval already finds an article for 98% of answerable tickets. And I kept the revised classification prompt after its hypothesis failed.

SECTION 10

Conclusions

CloudServe asked for a chatbot and I built an assistant for its agents. The discovery evidence favoured that: most of the answers already exist, the expensive tickets are the ones a machine should not handle alone, and the head of support named a confident wrong answer as the way the project fails. On the validation set the assistant labelled intent correctly for 91.3% of tickets, retrieved an expected article for 98.1% of the answerable ones, escalated all 14 tickets that policy sends to a specialist, kept a decision log that matched the tickets processed, and failed safely in each condition I could induce. It never sends anything to a customer.

What it achieves for CloudServe is a different question, and the evidence does not answer it. Nothing was tested with agents on real tickets, so there is no first contact resolution, reply time or satisfaction result. The hallucination rate and citation accuracy are unmeasured. Latency is well over target under free-tier limits. The hidden set has not been run. Appendix E (Tables E1 and E2) lists these and the smaller items that are still open.

10.1 What I would do next

1. **Measure the drafts.** Run the two-assessor review of 50 drafts with agreement reported. The packet, blank score sheets and scoring script are in the repository (*evaluation/human_review*).
2. **Fix the two draft problems.** Treat a subject that conflicts with the body as unconfirmed, and compare an article's plan scope with the customer's plan. Both are written into the requirements, and checking them needs a fresh set of tickets.
3. **Close the guardrail gaps.** Add the commitments check (no promises about refunds or timelines), widen the private-data patterns beyond credentials, and test paraphrased injection attempts.
4. **Build a live intake and a runtime kill switch,** with a named operator and a test that it works.
5. **Run a supervised pilot** with a few Tier One agents on real tickets, recording review time, edits, rejections, first contact resolution and satisfaction, split by tier and fluency. Enterprise customers already have the lowest first contact resolution in the data, so they need particular attention.
6. **Measure latency without pacing,** on a key with higher limits, recording where each ticket's time goes.

DECLARATION

Declaration of AI tool use

I used Claude Code, Anthropic's command-line coding assistant running Claude models, throughout this project, and the use was substantial. Working from my instructions, it wrote and edited code in the repository, including the pipeline, guardrails, evaluation harness, review application and tests. It wrote the analysis scripts for calibration, error analysis, confidence intervals and the fairness audit, ran the experiments and evaluation runs, and read the 50 validation drafts against the cited articles. It drafted the analysis documents in the repository (the development error analysis, the governance documents, the credential scan) and this report, and it drafted text in the stage workbooks and the effort log, including the estimated hours. Besides the assistant I relied on the supplied course pack (transcripts, data sets, specifications and templates), on open-source libraries (Chroma, a LangChain text splitter, the MiniLM sentence embedding model, FastAPI, requests, python-dotenv and python-docx), and on Qwen 3.8 27B, served by Groq, as the language model inside the system. The prompts that run in the system were drafted with the assistant and are listed in Table A1 (Appendix A).

The course asks that the problem statement, the interpretation of results and the reflection come from my own judgement. The problem statement in Section 3 is the text from my discovery workbook. The interpretation and reflection in Sections 7, 9 and 10 were drafted by the assistant from my workbooks and the result files. Every number in this report comes from a file in the repository.

There were places where I overrode or corrected the assistant. It told me a validation run did not exist when I had in fact started one, and I corrected that. The course documents gave the wrong deadline and I corrected it to 22 November. I decided not to carry out the two-assessor review of the drafts for lack of time, although the assistant had built the review packet, so the hallucination rate and citation accuracy are reported as not measured. I chose which of its suggested guardrail changes to build (the input check and the logging of classifier failures) and which to leave as known gaps. Its first guardrail demonstration used an invented key that resembled a real one, and the repository host's secret scanner rejected the push, so I approved rewriting my unpushed commits to remove it; a scan of the whole history found no real credential. The hours in the effort log are estimates that the assistant produced from the commit history and plan, and not a timesheet.

APPENDIX A

Appendix A. Prompt register

Table A1. Prompt register: purpose, version history and the version in use

ID	Purpose	Versions	In use
PR-01	Design-time prompt used while planning	V1.0, no later revisions	No
PR-02	Grounded reply drafter: answer only from the retrieved passages, cite each claim inline, say when evidence is insufficient	V1.0 initial. V2.0 (3 November 2026): inline citations and the structured list must match; stored citations come only from inline identifiers checked against the retrieved sources	v2.0
PR-03	Classifier: intent from 22 labels, urgency, confidences, urgency evidence excerpt, ambiguity flag	V1.0 initial. V2 (1 November): evidence excerpt or ambiguity flag required. V3 (4 November): active-problem wording; on 16 development tickets urgency matched 10 of 15 for both v2 and v3, so not adopted. V4 (6 November): intent definitions and tie-break rules; development intent accuracy 85.6% to 93.2%; default since 9 November (H1 not fully met)	v4
PR-04	Escalation summary prompt	V1.0. The implemented summary is built from the ticket, history and sources with no model call	No

Appendix B. Full result tables

Table B1. Intent classification per class, validation set (80 tickets, final run)

Intent	Tickets	Precision	Recall
account access	4	75%	75%
api key issue	6	100%	33%
api usage question	4	100%	100%
authentication failure	3	67%	67%
billing query	10	91%	100%
compliance request	1	100%	100%
configuration help	3	75%	100%
data export	1	100%	100%
data residency	6	100%	100%
database issue	1	100%	100%
deployment failure	5	100%	100%
feature request	3	100%	100%
integration help	3	100%	100%
onboarding	4	100%	100%
performance degradation	3	100%	100%
quota or overage	2	100%	50%
rate limit	1	100%	100%
rollback request	6	100%	100%
security incident	4	57%	100%
sso configuration	1	100%	100%
unclear request	6	100%	100%
webhook issue	3	100%	100%

Macro-averaged precision is 0.94 and recall 0.92. The four classes below 85% precision (account access, authentication failure, configuration help, security incident) each have 3 or 4 tickets. Security incident precision is low because some tickets that carry other labels were predicted as security incidents, which escalate.

Table B2. Routing on validation: route the labels expect against route taken

	Escalated by system	Sent to review	Auto-responded
Labelled escalate (32)	14	18	0
Labelled auto-respond (48)	3	45	0

Table B3. Outcomes of the 80 validation tickets

Outcome	Tickets
Escalated, summary prepared, no draft	17
Draft ready for agent review	50
Note for a human: evidence insufficient	13
Blocked by a guardrail	0

Table B4. Latency by outcome, validation run

Group	Tickets	Median	95th percentile	Over 30 s
Escalated (classification only)	17	2.9 s	54.9 s	4
Draft ready	50	6.3 s	53.9 s	14
Needs human	13	6.1 s	57.1 s	3
All	80	6.3 s	55.8 s	21

Appendix C. Requirement traceability

Table C1. Each requirement traced to the code that implements it and the tests that cover it

Requirement	Code	Tests
FR-01 ticket and context	src/tickets.py, src/review_store.py	test_tickets, test_agent_review_api
FR-02 documentation	src/retrieve.py	test_retrieve
FR-03 grounded draft	src/answering.py, src/guardrails.py	test_answering
FR-04 agent review, no send path	src/app.py, src/review_store.py, static/review.html	test_agent_review_api
FR-05 insufficient evidence, must-escalate intents	src/routing.py, src/guardrails.py	test_classification_routing, test_answering
FR-06 escalation summary	src/escalation.py	test_escalation
FR-07 sources and decisions recorded	src/logging_store.py, src/pipeline.py, src/review_store.py	test_logging_store, test_answering
FR-08 outcomes and feedback	src/outcome_store.py, src/dashboard.py	test_outcomes, test_dashboard
FR-09 intent, urgency, confidence	src/classification.py, src/routing.py, src/calibration.py	test_classification_routing, test_calibration, test_review_urgency
NFR-01 latency	src/evaluation.py	test_evaluation
NFR-04, NFR-05 privacy, auditability	src/guardrails.py, src/logging_store.py	test_answering, test_logging_store
NFR-06 fairness	scripts/analyze_accuracy_uncertainty.py, scripts/governance_fairness_audit.py	run on result files
NFR-08 evaluation integrity	src/scenarios.py, src/evaluation.py	test_calibration, test_evaluation

Appendix D. Workbooks and reproduction

Submitted with this report. The five stage workbooks (discovery, requirements version 2.0, prompt library, sprint plan, revision log) and the effort log in the workbooks folder, and the full repository in the source code folder. Supporting documents in the repository: *docs/governance.md*, *docs/governance_coverage_check.md*, *docs/validation_draft_review.md*, *docs/validation_latency_and_fairness.md*, *docs/development_error_analysis.md*, *docs/accuracy_uncertainty_and_fairness.md*, *docs/urgency_and_label_noise.md* and *docs/credential_scan.md*.

Evaluation runs. First validation run, 1 November 2026: 6 of 80 tickets before rate limiting. Paced run, 2 November: 80 of 80, prompt PR-03 v2, intent 88.8%. Final run in the final configuration: complete by 10 November, 80 of 80 with no errors, intent 91.3%, report regenerated 11 November. Hidden 120-ticket set: not run by me, zero times.

Reproduction. Install the requirements, copy *.env.example* to *.env* and add a model key. Run *python -m pytest tests -v* for the tests. Run *python scripts/run_evaluation.py --input <tickets.json> --output <results.jsonl> --processor src.pipeline:process_ticket* for an evaluation, adding *--resume* to continue an interrupted one, and *uvicorn src.app:app* for the review page. The demonstrations are *scripts/demo_guardrail_ticket.py* and *scripts/demo_failure_modes.py*.

Appendix E. Open items and uncertainties

This appendix collects what I am not sure of, what I estimated, and what I have not yet checked. None of it is used in the main text as if it were established. Table E1 lists the checks not yet made and Table E2 the estimates and judgements.

Table E1. Not yet checked

Item	State
Clean checkout (A1)	I have not yet cloned the repository into an empty directory on a second machine and followed the README step by step.
Deterministic routing (A5)	The router is a pure function and the model is called at temperature 0, but I did not run the same live ticket twice to confirm identical decisions.
Hidden evaluation set	Not provided to me and not run.
Kill switch	Restart-to-stop was not tested as a switch.

Table E2. Estimates, inferences and judgements

Item	What is uncertain
Hallucination and citation accuracy	Not measured. As a stand-in, one reader (the AI assistant, not a person) read the 50 validation drafts against the cited articles: 39 were sound, 7 contained a small assumption or embellishment, and 4 would mislead a reviewer who only skims. That is a read, not a rate. It cannot support a claim against the 5% and 95% targets.
Cause of the slow tickets	21 of 80 tickets took 42 to 62 seconds in every outcome group. I believe they waited on the provider's rate limit, but I did not log the cause, so this is an inference.
Fairness conclusions	Groups of 8 to 61 tickets cannot show a gap under about 20 points. "No evidence of a gap" is weaker than "no gap". Fairness of draft correctness and tone was not assessed.
Does the prompt gain generalise?	The development gain came from roughly nine scenarios. On validation the two prompts differ by two tickets. I have no further unseen data to test another change.
Why identical texts carry different urgency labels	Unknown. The supplied context (tier, fluency, channel, region) does not explain it. It may reflect customer circumstances that the text does not show.
Free-tier model	I do not know whether Qwen 3.8 27B on Groq's free tier behaves like the model CloudServe would run.
Risk ratings	The likelihood and impact ratings in Table 7 are my own judgement, not measurements.
Effort log	Hours and daily entries are estimates reconstructed on 11 November from commit dates and project notes (about 80 hours in total to that date). Some task dates are approximate.
Agent behaviour	Whether agents would catch a misleading draft is untested, because there was no pilot.